Kubernetes + Helm + Dorarion

> Beta baseline note (2026-10-10): current core installs two consolidated migrations. Replacing an earlier beta DB requires a new empty DB or an explicit reinstall that discards old data. Do not delete only the SQLx ledger. Use a fresh image built from this source. The README contains the current policy; an earlier downloaded PDF may describe the previous chain.

Use a dedicated HTTPS hostname with a root path. Sub-path installations are not supported. For the disposable plaintext PostgreSQL example only, the following Helm arguments explicitly allow insecure database transport. External production databases require sslmode=verify-full and an approved CA.

This guide matches the current source checkout. Build and verify a matching core image before installation; older published chart/image versions do not contain these fixes. PostgreSQL 16+ is required.

Simple cluster installation guide

> **Goal:** Install Dorarion from the checked-out Helm chart, connect it to an external PostgreSQL database, create the required Kubernetes Secret, create the first administrator, and verify the application.

- Run all commands from a terminal configured to access your Kubernetes cluster.
- Run from the root of this updated `dorarion-helm` checkout.
- Dorarion requires Kubernetes 1.28 or newer and Helm 3.
- PostgreSQL is not installed by this chart; prepare a database and backups first.
- A PDF version is also available.

1. Check Kubernetes and Helm

Confirm that `kubectl` points to the intended cluster before creating anything:

```
kubectl config current-context
kubectl cluster-info
kubectl version
helm version
```

Do not continue if the context names the wrong cluster. Dorarion requires a Kubernetes server version of at least 1.28.

2. Check the local chart and matching image

Inspect this checkout and select an image built from the matching core source:

```
helm show chart .

read -r -p "Verified matching image repository: " DORARION_IMAGE_REPOSITORY
read -r -p "Verified image digest (sha256:...): " DORARION_IMAGE_DIGEST
```

The chart metadata is:

- Chart: `dorarion`
- Chart version: `0.0.3`
- Application version: `0.0.3`
- Application image: your verified repository and immutable digest.

3. Prepare PostgreSQL

Create an empty PostgreSQL database and a dedicated Dorarion user. The user must be allowed to connect, create and alter tables, indexes, and constraints, and run Dorarion's migrations.

Use your database provider's TLS requirements in the connection URL. A typical URL has this form:

```
postgres://USER:PASSWORD@HOST:5432/DB_NAME?sslmode=verify-full
```

Back up the database before every Dorarion upgrade. Test restoration separately; the Helm chart does not operate or back up PostgreSQL.

Optional: run a simple PostgreSQL pod

If you do not already have PostgreSQL, this repository provides a single-node PostgreSQL example. It creates a 10 Gi persistent volume claim, one PostgreSQL Deployment, and an internal ClusterIP Service.

> **This example is for learning and small test installations.** It is not highly available, does not configure TLS, and does not create backups. Use a managed database or a maintained PostgreSQL operator for production.

Create the namespace and a random PostgreSQL password:

```
kubectl create namespace dorarion --dry-run=client -o yaml | kubectl apply -f -

POSTGRES_PASSWORD="$(openssl rand -hex 32)"

kubectl -n dorarion create secret generic dorarion-postgresql \
  --from-literal=POSTGRES_DB=dorarion \
```

```
  --from-literal=POSTGRES_USER=dorarion \
  --from-literal=POSTGRES_PASSWORD="$POSTGRES_PASSWORD"
```

Deploy the checked-out PostgreSQL example and wait until it is ready:

```
kubectl apply -f examples/postgresql.yaml

kubectl -n dorarion rollout status deployment/dorarion-postgresql \
  --timeout=5m
kubectl -n dorarion get pod,service,persistentvolumeclaim \
  -l app.kubernetes.io/name=dorarion-postgresql
```

If you opened a new terminal after creating PostgreSQL, recover the password into a shell variable without displaying it:

```
POSTGRES_PASSWORD="$(kubectl -n dorarion get secret dorarion-postgresql \
  -o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 --decode)"
```

For this example, set the database URL used in the next section:

```
DATABASE_URL="postgres://dorarion:${POSTGRES_PASSWORD}@dorarion-
postgresql.dorarion.svc.cluster.local:5432/dorarion"
DORARION_DEMO_HELM_ARGS=(--set extraEnv[0].name=DORARION_ALLOW_INSECURE_DATABASE --set-
string extraEnv[0].value=true)
```

4. Create the namespace and Secret

The Secret must exist before `helm install` because database migration runs as a pre-install hook.

Create the namespace:

```
kubectl create namespace dorarion --dry-run=client -o yaml | kubectl apply -f -
```

Generate the application secrets in your terminal and create the Kubernetes Secret. If you used the PostgreSQL example, `DATABASE_URL` is already set. For an external database, replace and run this first:

```
DATABASE_URL='postgres://USER:PASSWORD@HOST:5432/DB_NAME?sslmode=verify-full'
DORARION_DEMO_HELM_ARGS=()
```

Then create the application Secret:

```
: "${DATABASE_URL:?Set DATABASE_URL before continuing}"
JWT_SECRET="$(openssl rand -hex 48)"
SECRET_ENCRYPTION_KEY="$(openssl rand -base64 32)"
DORARION_BOOTSTRAP_PASSWORD="$(openssl rand -hex 24)"
```

```
kubectl -n dorarion create secret generic dorarion-secrets \
  --from-literal=DATABASE_URL="$DATABASE_URL" \
  --from-literal=JWT_SECRET="$JWT_SECRET" \
  --from-literal=SECRET_ENCRYPTION_KEY="$SECRET_ENCRYPTION_KEY" \
  --from-literal=DORARION_BOOTSTRAP_PASSWORD="$DORARION_BOOTSTRAP_PASSWORD"

unset DATABASE_URL POSTGRES_PASSWORD JWT_SECRET SECRET_ENCRYPTION_KEY
DORARION_BOOTSTRAP_PASSWORD
```

> **Protect these values.** Never commit them to Git, store them in Helm values, or post them in chat, screenshots, or tickets. Back up `SECRET_ENCRYPTION_KEY` in a password manager or secret manager. Losing it makes Dorarion's directly stored integration credentials unrecoverable.

Check only the Secret's key names, without printing their values:

```
kubectl -n dorarion describe secret dorarion-secrets
```

It must contain `DATABASE_URL`, `JWT_SECRET`, `SECRET_ENCRYPTION_KEY`, and `DORARION_BOOTSTRAP_PASSWORD`.

5. Install Dorarion and create the first administrator

Choose the real HTTPS address users and CI runners will use. Then install Dorarion and enable administrator bootstrap for this first installation only:

```
helm upgrade --install dorarion \
  . \
  "${DORARION_DEMO_HELM_ARGS[@]}" \
  --namespace dorarion \
  --set-string image.repository="$DORARION_IMAGE_REPOSITORY" \
  --set-string image.digest="$DORARION_IMAGE_DIGEST" \
  --set publicUrl=https://dorarion.example.com \
  --set bootstrapAdmin.enabled=true \
  --set bootstrapAdmin.username=admin \
  --set bootstrapAdmin.email=admin@example.com \
  --wait \
  --timeout 15m
```

Use an address reachable by users, GitLab runners, and other systems that send reports or webhooks to Dorarion. Production installations should use HTTPS.

The installation performs these operations in order:

1. runs the blocking database migration Job;
2. creates the Dorarion Deployment and Service;
3. waits for the application to become ready;
4. runs the first-administrator bootstrap Job.

6. Save the administrator password

Bootstrap reads DORARION_BOOTSTRAP_PASSWORD from the pre-created Secret (at least 16 characters); it never prints the password. Save it in your secret manager before unsetting the shell variable. Retrieve it from that manager or the Secret for login. The completed Job can be removed after verification. Never enable bootstrap again for an existing installation.

```
kubectl -n dorarion delete job dorarion-bootstrap-admin
```

7. Check the installation

Check the Helm release and Kubernetes workloads:

```
helm status dorarion --namespace dorarion
kubectl -n dorarion get deployment,pods,service,jobs
kubectl -n dorarion rollout status deployment/dorarion
```

If the migration fails, inspect its retained Job and logs:

```
kubectl -n dorarion get jobs
kubectl -n dorarion logs job/dorarion-migrate
```

The most common causes are an incorrect `DATABASE_URL`, blocked network access to PostgreSQL, missing Secret keys, or insufficient database permissions.

8. Open Dorarion locally

For a quick check without an Ingress, forward the ClusterIP Service:

```
kubectl -n dorarion port-forward service/dorarion 3000:80
```

Keep that terminal open, then visit:

http://127.0.0.1:3000/login

Sign in with `admin` or `admin@example.com` and the password saved in Step 6. In another terminal, verify readiness:

```
curl --fail http://127.0.0.1:3000/health/ready
```

9. Configure an Ingress

An Ingress controller and DNS must already exist. Create a local, non-secret values file named `values.local.yaml`:

```
publicUrl: https://dorarion.example.com

ingress:
  enabled: true
  className: nginx
  hosts:
    - host: dorarion.example.com
      paths:
        - path: /
          pathType: Prefix
  tls:
    - secretName: dorarion-tls
      hosts:
        - dorarion.example.com
```

Apply it without enabling administrator bootstrap:

```
helm upgrade dorarion \
  . \
  "${DORARION_DEMO_HELM_ARGS[@]}" \
  --namespace dorarion \
  --reuse-values \
  --set bootstrapAdmin.enabled=false \
  --values values.local.yaml \
  --wait \
  --timeout 15m
```

Configure the TLS Secret according to your Ingress controller or certificate manager. Do not add TLS private keys to a values file.

10. Upgrade Dorarion

Before an upgrade:

1. back up PostgreSQL and test that the backup can be restored;
2. back up the exact `SECRET_ENCRYPTION_KEY` and its key reference;
3. read the release notes and migration compatibility policy;
4. select a new immutable chart and application version.

Then upgrade, keeping administrator bootstrap disabled:

```
helm upgrade dorarion \
  . \
  --version NEW_CHART_VERSION \
```

```
  --namespace dorarion \
  --reuse-values \
  --set bootstrapAdmin.enabled=false \
  --wait \
  --timeout 15m
```

The pre-upgrade migration changes the database forward. Rolling back the Helm release does not reverse a database migration.

11. Uninstall boundaries

Remove the Helm release with:

```
helm uninstall dorarion --namespace dorarion
```

This does not remove the external PostgreSQL database or the manually created `dorarion-secrets` Secret. Preserve both when you may reinstall or restore. Delete the namespace only when you deliberately want to remove all remaining namespace resources.

If you deployed the sample PostgreSQL resources, they are also separate from the Helm release. Removing its persistent volume claim deletes the database when the storage class uses the usual `Delete` reclaim policy:

```
kubectl -n dorarion delete deployment,service dorarion-postgresql
kubectl -n dorarion delete persistentvolumeclaim dorarion-postgresql
kubectl -n dorarion delete secret dorarion-postgresql
```

Do not run the persistent volume claim deletion unless you intentionally want to destroy the sample database and have any required backup.

Quick recap

- Kubernetes context and version checked.
- Published OCI chart checked on Docker Hub.
- External PostgreSQL database and backups prepared.
- Or, for testing only, the single-node PostgreSQL example deployed.
- Namespace and required Kubernetes Secret created.
- Dorarion installed with first-install bootstrap enabled once.
- Pre-created administrator password stored in the secret manager; bootstrap logs contain no password.
- Deployment, readiness, and browser access verified.
- Future upgrades configured with bootstrap disabled.

Official references

- Dorarion Helm source
- Dorarion chart on Docker Hub
- Install Helm
- Helm OCI registries
- Install kubectl

Installation and upgrades — reviewed core

These settings describe the reviewed pre-1.0 core source. Select an image built from that source; an existing v0.0.3 registry tag does not prove it contains the remediation changes.

Pre-1.0: use the API contract shipped with the selected image. The reviewed source implements CI/CD, jobs, incidents and daily/weekly activity reporting.

- Generate `JWT_SECRET` (at least 32 bytes) and `SECRET_ENCRYPTION_KEY` (Base64 encoding of 32 random bytes) once. Keep both outside version control.
- The API pool defaults to 15 connections; allow another six worker connections per replica and temporary migration connections.
- Compose trusts only its exact internal `postgres` host. External PostgreSQL needs `sslmode=verify-full` and the trusted CA.
- Use `env://DORARION_EXT_<NAME>` for external credentials. Core secret variables are always denied.
- Argo CD needs a separately rotated application token and an active application/integration. Incident automation needs a repository-scoped incident source.

`/health/live` checks the process; `/health/ready` checks PostgreSQL. Worker health is separate at authenticated `/health/workers`. For Prometheus, store a 32–512-character ASCII `DORARION_METRICS_SCRAPE_TOKEN` in a Secret and use it only at `/metrics`. Configure exact trusted proxy networks before honoring forwarded client addresses.

Back up PostgreSQL and the exact encryption key before forward migrations. Run the migration Job/service once and disable server startup migrations. Change the generated bootstrap password on first login. Rotate Argo application tokens after upgrading. Audit retention cannot be shorter than 365 days. Weekly reports cover the seven complete local days before sending; reuse the same Idempotency-Key when retrying one manual send. Incident reopening must be explicit. Read the bundled API contract: demo routes were removed and unknown request fields are rejected. Validate backups, provider delivery and readiness in your environment before production rollout.

Helm: `database.connectionPoolSize=15`, `database.statementTimeoutMs=30000`, `database.trustedHosts=[]`, `security.secretEnvAllowlist=[]`, `security.trustedProxies=[]`, `security.reportRecipientDomains=[]`, `security.readmeImageOrigins=[]`. Optional monitoring credentials belong in the existing Secret; select their key with `security.metricsScrapeTokenSecretKey`. Hook Jobs receive the same policy and `extraEnvFrom` as the server.